



Entendendo transaction wraparound no Postgres

Israel Barth Rubio,
Senior Staff SDE II
04/09/2026

Agenda

01 Fundamentos e modelo mental

02 Transaction wraparound

03 O mecanismo de freezing

04 Remediando o problema

05 Protegendo o cluster

06 Perguntas

Fundamentos e modelo mental

Como o Postgres enxerga o tempo e a visibilidade das tuplas

MVCC (Multi-Version Concurrency Control)

01 Conceito

Leituras **não bloqueiam** escritas, e vice-versa.

02 Mecânica

Operações de escrita não sobrescrevem dados no disco; criam **novas versões de tuplas** mantendo as versões anteriores.

03 Visibilidade

Determinantes de visibilidade incluem **Transaction ID (xid)**, isolation level da sessão e o commit log.

Transaction ID (XID)



Representação no disco

Inteiro sem sinal de **32 bits**.



Espaço de endereçamento

Limite de $2^{32} \approx 4\text{B}$ transações.

psql - postgres

```
postgres=# BEGIN;  
BEGIN
```

```
postgres=# SELECT pg_current_xact_id();  
pg_current_xact_id
```

```
-----  
100  
(1 row)
```

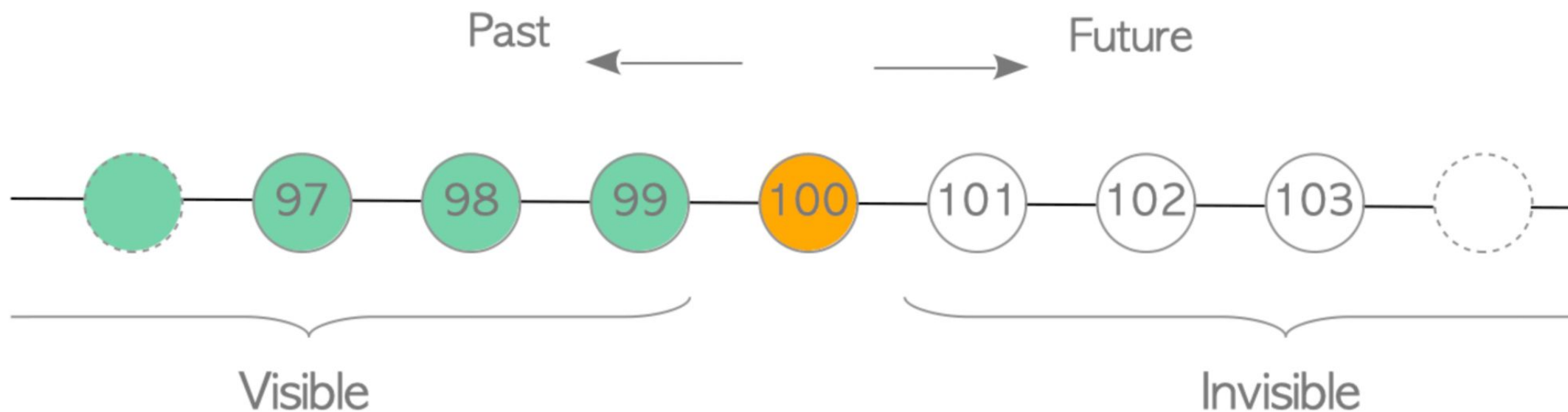
Transaction ID (XID)

Controle de visibilidade

Usado diretamente para determinar se uma versão de tupla é visível para transações concorrentes.

Espaço de endereçamento ideal

Em um mundo ideal, as transações avançariam em um espaço serial contínuo e infinito.



Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-01.png>

Transaction ID (XID)

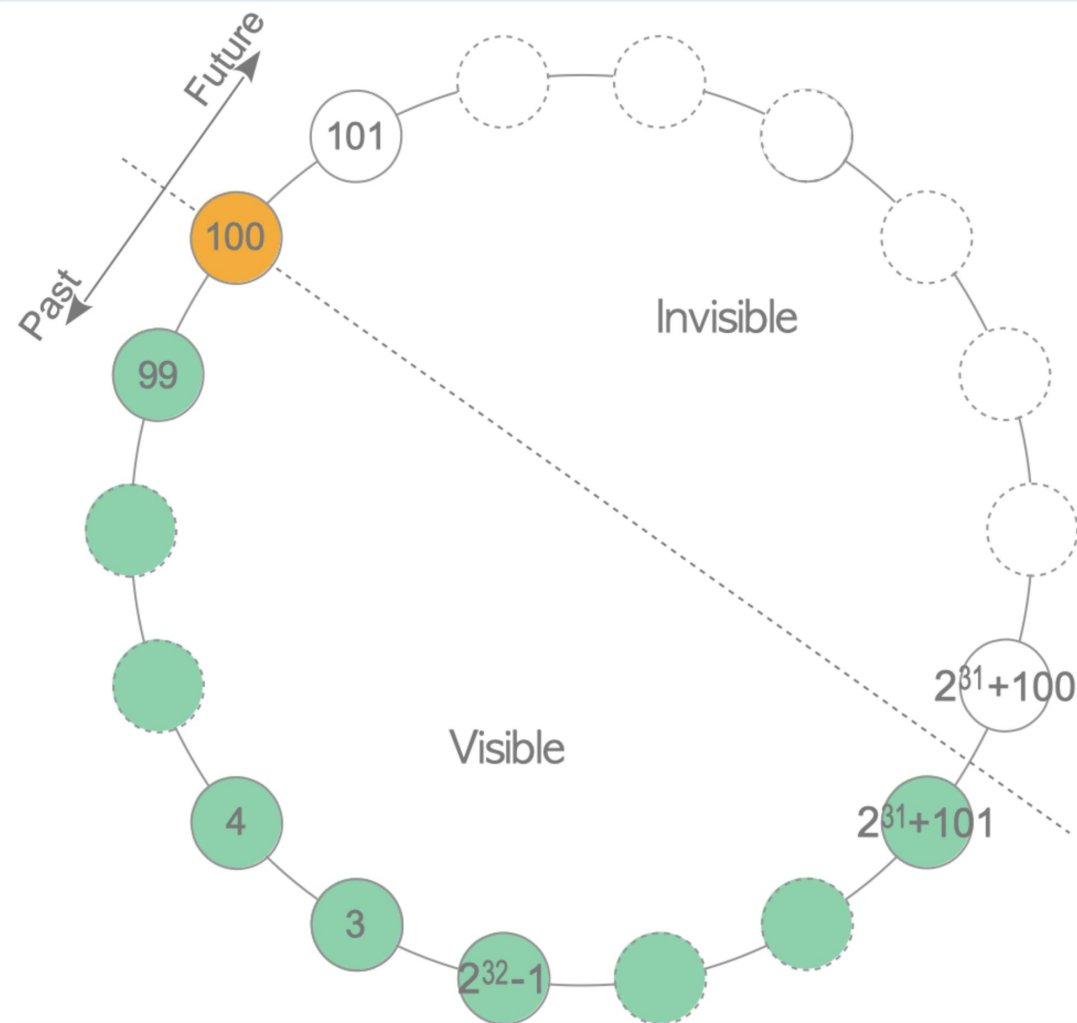
🔄 Espaço circular (32 bits)

Limitação prática de representação por um inteiro de 32 bits. O endereçamento **reinicia de forma circular** ao esgotar o espaço.

🕒 Divisão do espaço

Espaço total **dividido simetricamente** em relação ao XID atual:

- **Passado (2^{31} XIDs):** Transações anteriores que já foram confirmadas e são visíveis.
- **Futuro (2^{31} XIDs):** Transações futuras e ativas que são invisíveis.



Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-01.png>

Formato das tuplas



Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-02.png>

+ t_xmin

XID da transação que criou esta versão da tupla.

- t_xmax

XID da transação que removeu esta versão da tupla (**0** se for a versão ativa).

t_cid

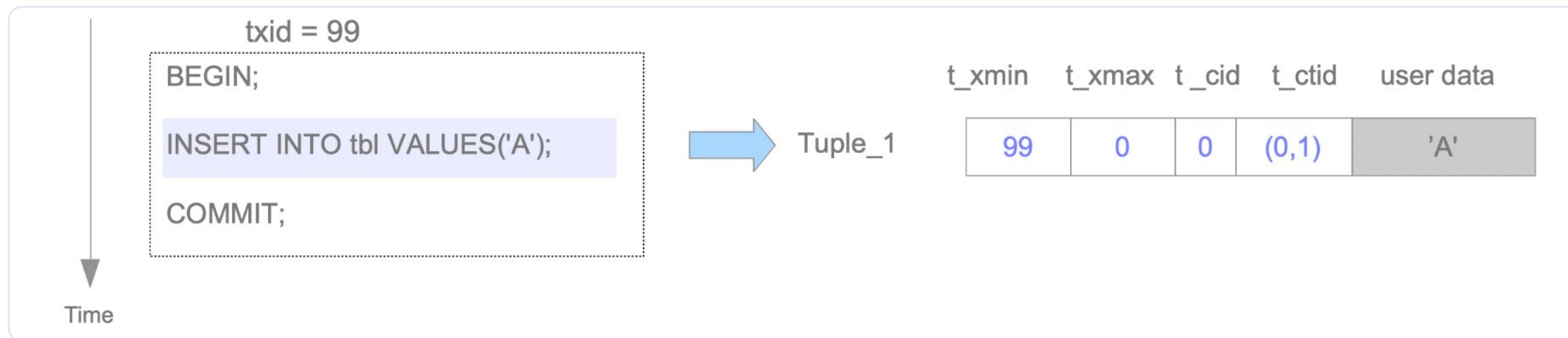
Sequência do comando dentro da transação, iniciando em **0**, que criou esta versão da tupla.

🔗 t_ctid

Ponteiro físico (**página, tupla**) para a própria tupla, se for a versão ativa, ou para a próxima versão.

Comportamento DML – INSERT

📘 INSERT insere uma nova versão na tabela (heap)



Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-04.png>

+ t_xmin: 99

O XID da transação que criou a versão da tupla.

- t_xmax: 0

A versão mais recente da tupla.

t_cid: 0

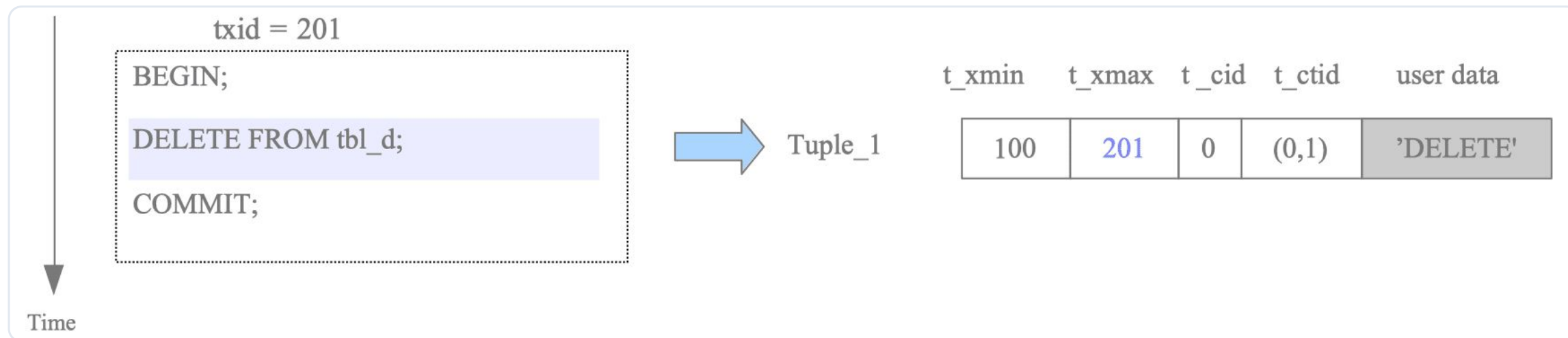
O primeiro comando dentro da transação.

↪ t_ctid: (0, 1)

A versão mais recente da tupla (aponta para si mesma).

Comportamento DML – DELETE

DELETE marca a versão como removida



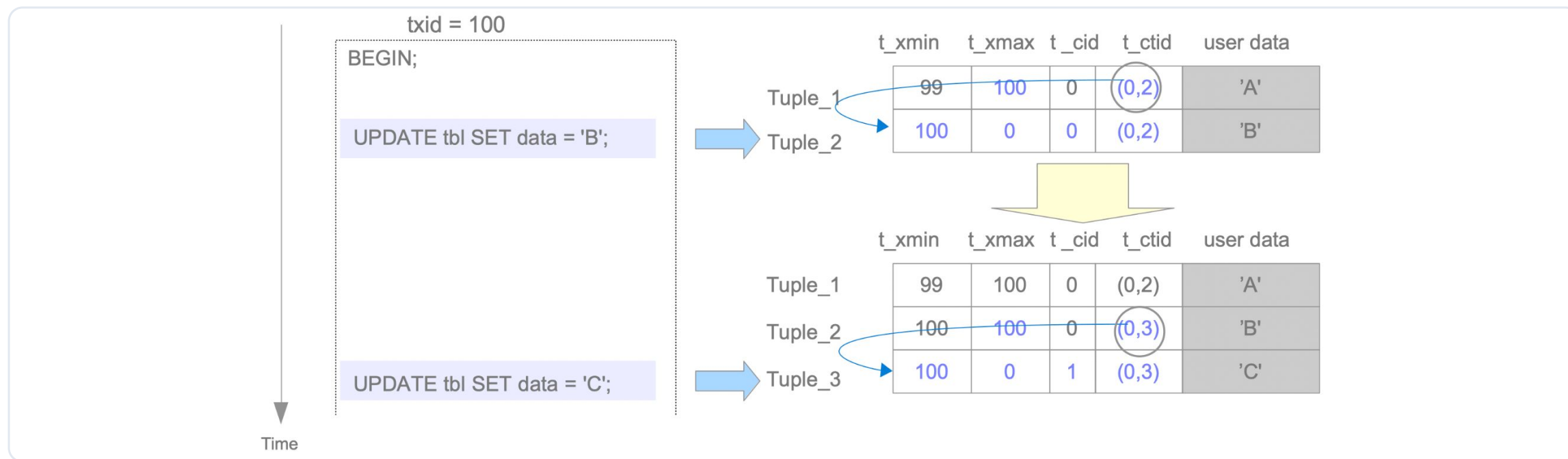
Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-05.png>

– **t_xmax: 201**

O XID da transação que removeu a versão da tupla.

Comportamento DML – UPDATE

UPDATE é implementado logicamente como um DELETE + INSERT



Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-06.png>

– t_xmax

Na versão antiga da tupla, setado com o XID corrente para remover a versão.

+ t_xmin

Na versão nova da tupla, setado com o XID corrente para criar a versão.

↔ t_ctid

Na versão antiga, aponta para a versão nova. Na versão nova, aponta para si mesma.

Estado das tuplas

Live tuples

Versões de tuplas visíveis para transações em andamento ou futuras.

Dead tuples

Versões antigas de tuplas atualizadas ou deletadas que não são mais visíveis para nenhuma transação ativa.

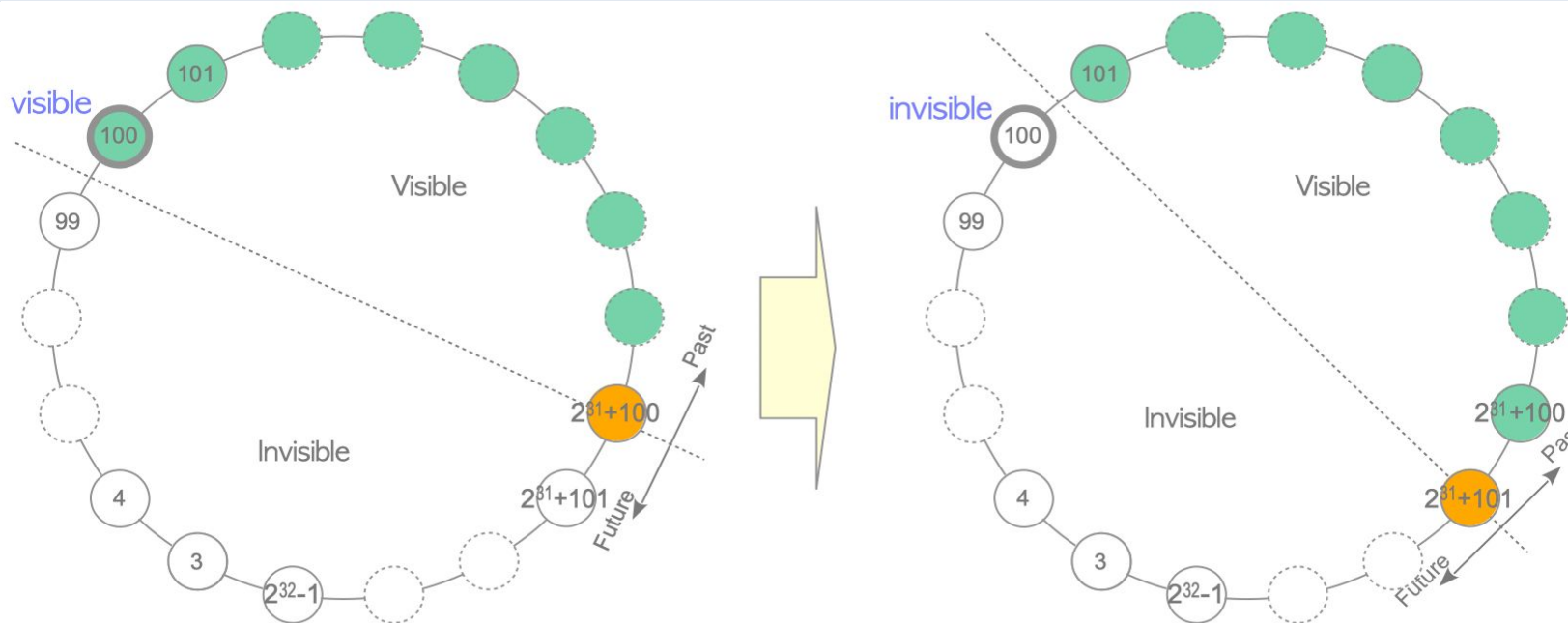
Limpeza física

Eventualmente serão removidas da tabela pelo vacuum (ou autovacuum).

Transaction wraparound

O risco de tuplas válidas antigas se tornarem invisíveis

Transaction ID wraparound



Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-21.png>

🔧 Mecânica da falha

Ocorre quando o XID atual do cluster avança **mais de 2^{31}** transações **além do xmin** de uma **live tuple**.

👁️ Impacto

Passa a enxergar a tupla válida como se estivesse no "futuro", **tornando-a invisível**.

⚠️ Consequência

Perda massiva de visibilidade de **dados válidos**.

O mecanismo de freezing

Como o VACUUM imuniza tuplas antigas contra a passagem do tempo

Mecanismo de freezing (congelamento)

	t_xmin	t_xmax	t_informask	user data		t_xmin	t_xmax	t_informask	user data
Tuple 1	99			'A'	Freeze	99		XMIN_FROZEN	'A'
Tuple 2	100			'B'		100		XMIN_FROZEN	'B'
Tuple 3	200000			'C'		200000			'C'
Tuple 4	1.5 million			'D'		1.5 million			'D'
Tuple 5	2.0 million			'E'		2.0 million			'E'

Fonte: <https://www.interdb.jp/pg/pgsql05/fig-5-22.png>

🎯 Objetivo

Impedir que tuplas antigas fiquem no **"futuro"** conforme o uso de XIDs avança no cluster.

✳ Mecânica

O **VACUUM** grava a flag **XMIN_FROZEN** na infomask da tupla, tornando-a permanentemente visível para qualquer transação.

🔗 xmin horizon

Limite do XID mais antigo. Nenhuma tupla mais recente que o **xmin horizon** pode ser congelada pelo processo.

Hierarquia de age (idade)

Idade da tupla

01

Métrica individual aplicada ao nível mais básico de armazenamento (tupla).

Cálculo: `current_XID - t_xmin`

Idade da tabela

02

Determinada pela tupla não congelada mais antiga presente na tabela.

Catálogo: `pg_class.relfrozenxid`

Idade do banco

03

Definida com base na idade da tabela mais antiga contida neste banco.

Catálogo: `pg_database.datfrozenxid`

Idade do cluster

04

Definida com base na idade do banco mais antigo do cluster.

Parâmetros de freezing

vacuum_freeze_min_age

Padrão: **50M**

Congela tuplas em páginas lidas pelo **VACUUM** se **$t_{xmin} < OldestXmin - vacuum_freeze_min_age$**

vacuum_freeze_table_age

Padrão: **150M**

Força o **VACUUM** a ler todas as páginas não congeladas da tabela.

autovacuum_freeze_max_age

Padrão: **200M**

Similar ao anterior, mas mais agressivo: não cancelável e é disparado mesmo se **autovacuum = off**.

vacuum_failsafe_age

Padrão: **1.6B**

Último recurso para evitar **XID wraparound** em um cluster ativo. Extremamente agressivo e focado.

Remediando o problema

O que trava o xmin horizon e
como agir quando o incidente
atinge a produção

Limitantes de avanço do xmin horizon

Transações longas

01

Sessões com queries longas ou no estado **idle-in-transaction** no cluster.

Standbys com feedback

02

Standbys com **hot_standby_feedback = on** e executando queries longas ou enfrentando delays.

Prepared transactions

03

Transações “2-phase commit” preparadas via **PREPARE TRANSACTION** e abandonadas ou esquecidas.

Replication Slots

04

Slots de replicação física ou lógica inativos ou atrasados.

Sintomas nos logs

Avisos iniciais

WARNING: oldest xmin is far in the past

WARNING: database "XXX" must be vacuumed within YYY transactions

WARNING: database with OID XXX must be vacuumed within YYY transactions

Erros críticos

ERROR: database is not accepting commands to avoid wraparound data loss in database "XXX"

ERROR: database is not accepting commands to avoid wraparound data loss in database with OID XXX

Procedimento de remediação



Modo single user

Se o cluster foi derrubado por erro crítico, reinicie em modo **single user** . 01
Isso permite executar queries de leitura e comandos de manutenção.

```
postgres --single -D pgdata_dir dbname
```



Liberar horizon

Identifique e remova os **fatores limitantes** que estão limitando o avanço do **xmin horizon** . 02



VACUUMs

Execute **VACUUMs direcionados** , priorizando resolver as tabelas mais antigas. 03

Primeiro estágio: liberar o xmin horizon

Visão geral de fatores limitantes

Velho em relação ao XID atual do cluster

```
SELECT -- Idade da sessão mais velha e ativa no cluster
(SELECT coalesce(max(age(backend_xmin)), 0)
 FROM pg_stat_activity) AS oldest_xact_age,
-- Idade da sessão mais velha e ativa em um standby desse cluster
(SELECT coalesce(max(age(backend_xmin)), 0)
 FROM pg_stat_replication) AS oldest_standby_xact_age,
-- Idade da prepared transaction mais velha no cluster
(SELECT coalesce(max(age(transaction)), 0)
 FROM pg_prepared_xacts) AS oldest_prepared_xact_age,
-- Idade do replication slot mais velho no cluster
(SELECT greatest(
        coalesce(max(age(xmin)), 0),
        coalesce(max(age/catalog_xmin)), 0)
)
FROM pg_replication_slots) AS oldest_replication_slot_age;
```

Identificar sessões limitantes

Se o problema for `oldest_xact_age`

```
-- Retorna as 10 sessões mais velhas que já cruzaram autovacuum_freeze_max_age
-- Quanto mais próxima a idade for de 2B, pior
SELECT pid,
       age(backend_xmin),
       datname,
       username,
       application_name,
       client_addr,
       state,
       backend_type,
       query
FROM pg_stat_activity
WHERE state <> 'idle'
      AND age(backend_xmin) > current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 2 DESC
LIMIT 10;
```

Terminar sessões limitantes

 Para cada pid da query anterior

```
-- Após avaliar o resultado, se possível, terminar os PIDs das sessões
-- problemáticas

-- Tente um cancelamento suave, usando "SIGINT":
SELECT pg_cancel_backend(pid);

-- Se não for possível, use "SIGTERM":
SELECT pg_terminate_backend(pid);
```

Identificar standbys limitantes

Se o problema for oldest_standby_xact_age

```
-- Retorna os 10 standbys mais velhos que já cruzaram autovacuum_freeze_max_age
-- Quanto mais próxima a idade for de 2B, pior
SELECT pid,
       age(backend_xmin),
       username,
       application_name,
       client_addr,
       state
FROM pg_stat_replication
WHERE age(backend_xmin) > current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 2 DESC
LIMIT 10;
```

Terminar sessões em standbys limitantes

Em cada standby da query anterior

O processo de remediação é o mesmo, mas as ações devem ser tomadas diretamente nos standbys que estão afetando o cluster:

🔍 Identificar sessões

01

Executar as queries de diagnóstico para localizar as conexões ativas no standby, que estão retendo o avanço do xmin no cluster

✖ Terminar sessões

02

Proceder com a terminação das sessões problemáticas no standby através de cancelamento suave (**pg_cancel_backend**) ou forçado (**pg_terminate_backend**).

Identificar prepared transactions limitantes

Se o problema for oldest_prepared_xact_age

```
-- Retorna as 10 prepared transactions mais velhas que já cruzaram
-- autovacuum_freeze_max_age
-- Quanto mais próxima a idade for de 2B, pior
SELECT transaction,
       gid,
       age(transaction),
       prepared,
       owner,
       database
FROM pg_prepared_xacts
WHERE age(transaction) > current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 3 DESC
LIMIT 10;
```

Encerrar prepared transactions limitantes

 Para cada gid da query anterior

```
-- Após avaliar o resultado, faça COMMIT ou ROLLBACK das prepared transactions
-- problemáticas usando o "gid" delas

-- Faça commit, se fizer sentido:
COMMIT PREPARED '<gid>';

-- Faça rollback, se fizer sentido:
ROLLBACK PREPARED '<gid>';
```

Identificar replication slots limitantes

Se o problema for oldest_replication_slot_age

```
-- Retorna os 10 replication slots mais velhos que já cruzaram
-- autovacuum_freeze_max_age
-- Quanto mais próxima a idade for de 2B, pior
SELECT slot_name,
       age(xmin),
       slot_type,
       database,
       temporary,
       active,
       active_pid
FROM pg_replication_slots
WHERE greatest(age(xmin), age(catalog_xmin)) >
current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 2 DESC
LIMIT 10;
```

Eliminar replication slots limitantes

Para cada slot_name da query anterior

```
-- Após avaliar o resultado, se necessário e possível, remova os replications  
slots limitantes usando o "slot_name" deles  
-- Lembre-se que isso destruirá a replicação dos clientes deles  
SELECT pg_drop_replication_slot(slot_name := '<slot_name>');
```

Segundo estágio: VACUUM direcionado para congelar tuplas antigas

Identificar bancos mais velhos no cluster

```
-- Retorna os 10 bancos de dados mais velhos do cluster que já cruzaram
-- autovacuum_freeze_max_age
-- Quanto mais próxima a idade for de 2B, pior
SELECT datname,
       age(datfrozenxid)
FROM pg_database
WHERE age(datfrozenxid) > current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 2 DESC
LIMIT 10;
```

Identificar tabelas mais velhas no banco

Em cada banco de dados da query anterior

```
-- Retorna as 30 tabelas mais velhas do banco de dados que já cruzaram
-- autovacuum_freeze_max_age
-- Quanto mais próxima a idade for de 2B, pior
SELECT oid::regclass AS table_name,
       age(relfrozenxid) AS age,
       pg_size_pretty(pg_total_relation_size(oid)) AS table_total_size
FROM pg_class
WHERE relkind IN ('r', 'm', 't')
AND age(relfrozenxid) > current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 2 DESC,
       pg_total_relation_size(oid) DESC
LIMIT 30;
```


VACUUM nas tabelas mais velhas

Em cada table_name da query anterior

```
-- Congele tuplas em cada tabela reportada pela query anterior  
VACUUM (INDEX_CLEANUP OFF) <table_name>;
```

Nota

Ao desabilitar **INDEX_CLEANUP**, permitimos que o **VACUUM** rode mais rápido durante a emergência.

Importante

Não utilizar a opção **FREEZE**, um **VACUUM** regular é suficiente.

- **FREEZE** é um VACUUM mais agressivo que roda com **vacuum_freeze_min_age = 0** e **vacuum_freeze_table_age = 0**
- Por consequência, **FREEZE** causa **I/O excessivo** e é muito mais lento.

Protegendo o cluster

Consultas essenciais e rotinas
para antecipar o risco e evitar
incidentes

Monitoramento

XIDs restantes até um erro crítico

📘 Executar em um banco do cluster é suficiente

```
-- Acompanhar quantas transações ainda podem ser executadas antes do cluster
-- ser derrubado para prevenir transaction wraparound
SELECT 2^31 - max(age(datfrozenxid)) AS xid_left
FROM pg_database;
```

Bancos mais velhos do cluster

📘 Executar em um banco do cluster é suficiente

```
-- Acompanhar quais bancos do cluster estão puxando o xmin horizon para longe
SELECT datname,
       age(datfrozenxid)
FROM pg_database
WHERE age(datfrozenxid) > current_setting('autovacuum_freeze_max_age')::bigint
ORDER BY 2 DESC
LIMIT 10;
```

Tabelas propensas a receber VACUUM

 Necessário executar em todos os bancos do cluster

```
-- Acompanhar quais tabelas do banco estão mais perto de receber um vacuum
-- preventivo
SELECT oid::regclass AS table_name,
       relfrozenxid,
       current_setting('autovacuum_freeze_max_age')::bigint - age(relfrozenxid)
AS xid_left_to_wraparound_vacuum,
       pg_size_pretty(pg_total_relation_size(oid)) AS table_total_size
FROM pg_class
WHERE relkind IN ('r', 'm', 't')
ORDER BY 3 ASC,
        pg_total_relation_size(oid)
LIMIT 10;
```

Recomendações

Recomendações



Monitoramento

01

Implemente monitoramento e alertas para evitar situações críticas.



Transações Curtas

02

Mantenha transações tão curtas quanto possível.

Provê uma série de benefícios, incluindo proteção contra transaction wraparound.



Parâmetros

03

Use parâmetros de auxílio como:

`idle_in_transaction_session_timeout`

`statement_timeout`



MultiXactID

04

Estude sobre MultiXactID (MultiXID) wraparound.

Conceitos parecidos com XIDs, mas focados em MultiXactID.

Perguntas?

LinkedIn



Slides

